

\$ Mini Shell em Python

Uma Exploração das Chamadas ao Sistema

Projeto Integrador de Sistemas Operacionais

Autores:

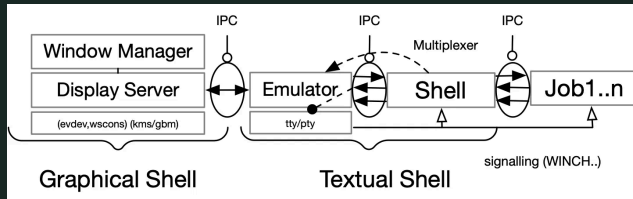
Pedro Afonso Braga

Risia Cerqueira

Universidade Federal da Bahia (UFBA)

Instituto de Computação

O Que é um Shell e o Contexto do Projeto



► Definição de Shell

Um interpretador de comandos que atua como interface entre o usuário e o Sistema Operacional.

► Objetivo do Projeto

Desenvolver um mini shell educativo para simular a execução de comandos em um terminal Linux.

► Foco

Prática com conceitos fundamentais de Sistemas Operacionais, como criação e controle de processos.

► Tecnologia

Implementação em Python, utilizando módulos de baixo nível (`os`, `shlex`).

A Base do Shell: Chamadas ao Sistema

os.fork()

Criação de um novo processo (filho) a partir do processo shell (pai). Essencial para a execução paralela de comandos.

os.execvp()

Substituição do processo filho pelo programa externo solicitado (ex: ls, cat, echo).

os.waitpid()

O processo pai aguarda o término da execução do processo filho, garantindo a ordem e o controle.

os.read() e os.write()

Manipulação de entrada e saída padrão (stdin, stdout, stderr) de forma direta, conforme requisito.

os.chdir()

Implementação do built-in cd para mudança de diretório.

```
#include<stdio.h>    // printf()
#include<stdlib.h>    // exit()
#include<sys/types.h> // pid_t
#include<sys/wait.h>  // wait()
#include<unistd.h>    // fork

int main(int argc, char **argv)
{
    pid_t pid;
    pid = fork();
    if(pid==0)
    {
        printf("It is the child process and pid is %d\n",getpid());
        int i=0;
        for(i=0;i<8;i++)
        {
            printf("%d\n",i);
        }
        exit(0);
    }
    else if(pid > 0)
    {
        printf("It is the parent process and pid is %d\n",getpid());
        int status;
        wait(&status);
        printf("Child is reaped\n");
    }
    else
    {
        printf("Error in forking..\n");
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

Arquitetura do Mini Shell: O Loop Principal

[`shell_loop()`]

O coração do shell, que se repete até o comando `exit` ser digitado.

[`read_line()`]

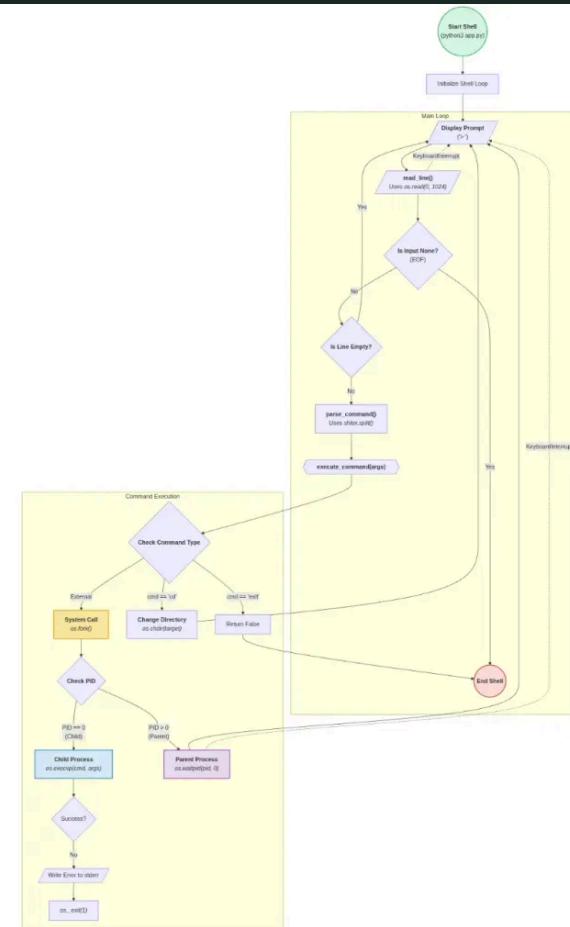
Exibe o prompt (`>`) e lê a entrada do usuário usando `os.read()`.

[`parse_command()`]

Processa a linha de comando, dividindo-a em argumentos usando `shlex.split()`.

[`execute_command()`]

Decide se o comando é um built-in (`exit`, `cd`) ou um comando externo.



Fluxo de Execução: Fork, Exec e Wait

1. `os.fork()`

Cria o processo filho. O processo pai recebe o PID do filho (> 0), enquanto o filho recebe 0.

2. Processo Filho (`PID == 0`)

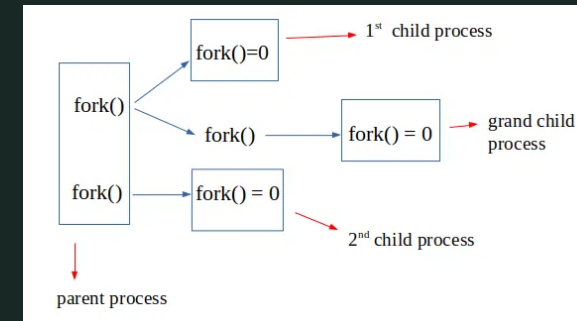
Tenta executar o comando com `os.execvp()`. Se falhar (ex: comando não encontrado), encerra com `os._exit(1)` e exibe mensagem de erro.

3. Processo Pai (`PID > 0`)

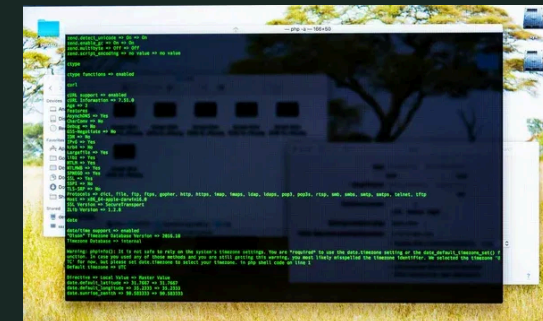
Aguarda o término do filho com `os.waitpid()`. Continua o loop do shell após o filho terminar.

Fluxograma Completo

O diagrama do mini-shell mostra este ciclo integrado ao loop principal, com tratamento de erros e built-ins.

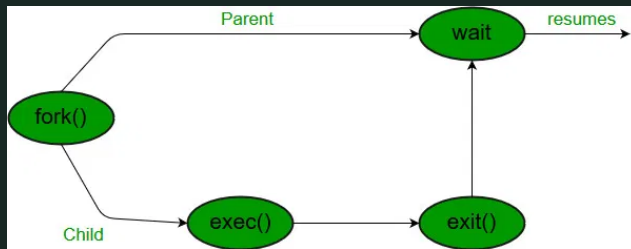


Árvore de Processos: Fork e Hierarquia



Transições de Estado: Parent/Child/Wait

Comandos Internos e Robustez



► Built-in exit

Encerra o loop principal do shell de forma controlada.

► Built-in cd

Implementado com `os.chdir()`, alterando o diretório de trabalho do próprio processo shell.

► Tratamento de Erros

Mensagens amigáveis para comandos não encontrados (via falha no `execvp`) e erros no `cd` (via `try/except` e `os.write(2, ...)`).

► Comandos Testados

`ls -l`, `echo Olá Mundo`, `cat arquivo.txt`

Limitações da Implementação Atual

- ▶ **Foco Educacional**

A implementação priorizou a clareza dos conceitos de chamadas ao sistema, mantendo o código simples e didático para fins de aprendizado.

- ▶ **Funcionalidades Ausentes**

Não implementa pipes (`|`), redirecionamentos (`>`, `<`) ou execução em background (`&`), que são recursos avançados.

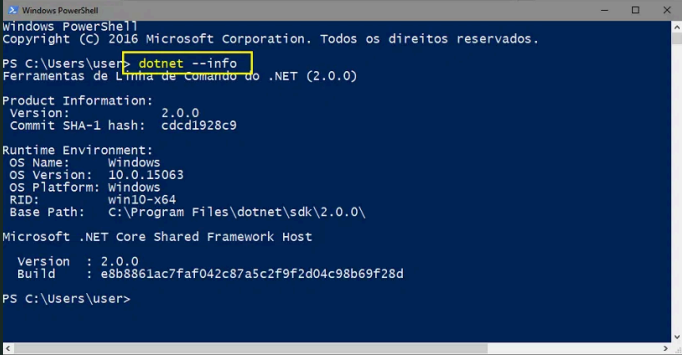
- ▶ **Parsing Simples**

Não suporta expansão de variáveis (`$VAR`) ou globbing (`*`), utilizando apenas `shlex.split()` para divisão de argumentos.

- ▶ **Desafio com `os.read()`**

A leitura com `os.read()` pode ter comportamento inesperado em cenários complexos de redirecionamento de entrada, com possível perda de buffer residual.

Demonstração: Mini Shell em Ação



```
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. Todos os direitos reservados.

PS C:\Users\user> dotnet --info
Ferramentas de Linha de Comando do .NET (2.0.0)

Product Information:
  Version:      2.0.0
  Commit SHA-1 hash: cdc1928c9

Runtime Environment:
  OS Name:      Windows
  OS Version:   10.0.15063
  OS Platform:  Windows
  RID:          win10-x64
  Base Path:    C:\Program Files\dotnet\sdk\2.0.0\

Microsoft .NET Core Shared Framework Host
  Version : 2.0.0
  Build   : e8b8861ac7faf042c87a5c2f9f2d04c98b69f28d

PS C:\Users\user>
```

Exemplo 1: Echo

```
> echo Olá Mundo
```

Exibe texto na saída padrão

Exemplo 2: Listagem

```
> ls -l
```

Lista arquivos do diretório corrente

Exemplo 3: Navegação

```
> cd /tmp
```

```
> pwd
```

Muda de diretório e exibe o caminho

Exemplo 4: Erro

```
> comando_inexistente
```

Demonstra tratamento de comando não encontrado

Conclusão e Principais Aprendizados

► Sucesso do Projeto

O projeto cumpriu com sucesso o objetivo de simular um shell, aplicando as chamadas ao sistema essenciais: `fork()`, `execvp()` e `waitpid()`. Todos os requisitos mínimos foram atendidos, incluindo a execução de múltiplos comandos com argumentos, tratamento de erros e modularização adequada do código.

► Aprendizados Principais

Compreensão prática e profunda do ciclo de vida de processos em um Sistema Operacional. Entendimento de como o kernel gerencia a criação, execução e finalização de processos, bem como a importância da sincronização entre processos pai e filho através de chamadas ao sistema.

► Próximos Passos

Implementação de funcionalidades avançadas como pipes (`|`), redirecionamentos (`>`, `<`) e execução em background (`&`). Expansão de variáveis e suporte a histórico de comandos também são melhorias viáveis para versões futuras do projeto.

Obrigado!

Obrigado pela atenção!

\$ Autores

Pedro Afonso Braga

Risia Cerqueira

Universidade Federal da Bahia (UFBA)

Instituto de Computação

Projeto Integrador – Mini Shell com Chamadas ao Sistema

Perguntas?